And there we have it; phpcs has parsed the file and found all of the PSR-12 violations it contains. CodeSniffer will also check multiple files in a single run when invoked with a directory path rather than a specific file. It is also highly configurable and customizable, with an XML-based config system that allows you to define your own rules (or use another project's rules). The full documentation can be found on the project's GitHub page: https://github.com/squizlabs/PHP_CodeSniffer/wiki.

Fixing Things

Note that toward the bottom of the output is a message about PHPCBF (PHP Code Beautifier and Fixer), another tool bundled with phpcs that allows for automatic fixing of standards violations. When we built the image and installed PHP_CodeSniffer via pear, it bundled the phpcbf PHAR as well. We can therefore repurpose our phpcs Docker image and have it run phpcbf by simply changing the container ENTRYPOINT, which can be done either with a new Dockerfile or with a simple command line option when running the container:

```
$ alias docker-phpcbf='docker run -it --rm -v "$PWD":/usr/src/myapp -w /
usr/src/myapp --entrypoint phpcbf local/php-codesniffer:8.4'
```

```
$ docker-phpcbf --standard=PSR12 ./phpcs_broken.php
PHPCBF RESULT SUMMARY
----------------------------------------------------------------------
FILE                                                  FIXED  REMAINING
----------------------------------------------------------------------
/usr/src/myapp/phpcs_broken.php                       7      5
----------------------------------------------------------------------
A TOTAL OF 7 ERRORS WERE FIXED IN 1 FILE
----------------------------------------------------------------------
```

The contents of phpcs_broken.php should now be as follows:

```
$num = 1 + 1 ;

class badClassName
{
    function BadMethodName($arg1, $arg2)
```